Sinais vitais: escolhas e limites de Random Forest e MLP

Mário Cordeiro Júnior[1], **André F. Maccarini**[1]

[1]Universidade Tecnológica Federal do Paraná (UTFPR)
Curitiba – PR – Brasil
Sistemas Inteligentes – Tarefa 2

Resumo. *Comparamos Random Forest e MLP para estimar a gravidade e a classe dos registros de sinais vitais fornecidos na atividade. Queremos entender como as duas técnicas aprendem e onde podem falhar em casos delicados. Usamos qPA, pulso e respiração. Separamos 900 registros para treinar e comparar configurações por validação cruzada e 600 para avaliar depois. Analisamos o erro na gravidade, os acertos e erros por classe e o efeito de dar mais peso aos erros com vítimas críticas. Essa prioridade é uma escolha nossa: deixar um crítico sem reconhecimento preocupa, mas um falso alerta também pode ocupar uma equipe necessária a outra vítima. Os resultados mostram o que aconteceu com os modelos e dados usados. Não bastam para decidir um resgate real. Nossa proposta é que o modelo sugira e a decisão final continue humana.*

Abstract. *We compare Random Forest and MLP to estimate severity and classify the vital-sign records provided for the assignment. We want to understand how both methods learn and where they can fail in sensitive cases. We use qPA, pulse and breathing rate, with 900 records for training and cross-validation and 600 for later evaluation. We examine severity errors, results for each class and the effect of giving more weight to errors involving critical victims. This priority is our choice: missing a critical victim is serious, but a false alert can also take a team away from another victim. The results describe these models and data. They are not enough to decide a real rescue. The model should suggest, while the final decision remains human.*

1. O que queremos analisar

A atividade pede estimar a gravidade e classificar os registros em quatro categorias: crítico, instável, potencialmente estável e estável. Também pede usar Random Forest e MLP, comparar os resultados e discutir as consequências para as pessoas [1]. Seguimos essas técnicas da disciplina, mesmo quando o resultado não é bom.

Queremos comparar os algoritmos e mostrar suas limitações em casos delicados. Nossa maior preocupação é deixar uma vítima crítica sem reconhecimento. Mas também precisamos olhar o outro lado: mandar uma equipe buscar alguém que não precisava daquela prioridade pode atrasar o socorro de outra vítima. Por isso, não achamos suficiente olhar só a porcentagem de acertos.

Os três sinais não mostram tudo o que importa num resgate. Por exemplo, não temos risco ambiental, acesso ao local, mudanças no estado da vítima ou equipes disponíveis. Escolhemos tratar a saída como uma sugestão. A decisão final é humana e precisa considerar informações que ficaram fora do modelo. Os erros deste exercício não permitem calcular quantas vidas seriam salvas ou perdidas.

2. Metodologia: como fizemos e por que escolhemos assim

2.1. Dados, entradas e saídas

Usamos os 1.500 registros da atividade. O enunciado informa que os valores foram gerados aleatoriamente dentro de intervalos. Então, são dados do exercício, não registros que comprovem o comportamento de pacientes reais. Cada linha tem um ID e uma medição, sem histórico da vítima. As entradas são qPA, pulso e respiração. Não usamos as pressões brutas porque o enunciado manda usar a qualidade da pressão, qPA. Nesse atributo, zero significa qualidade máxima, e não falta de medição.

O ID serve para identificar a linha, não para prever a saúde. A gravidade e a classe são as respostas que queremos aprender. Colocá-las nas entradas daria parte da resposta ao modelo. Para cada técnica, treinamos um modelo que estima o número da gravidade, chamado regressor, e outro que prevê a classe, chamado classificador. Prever a classe diretamente é uma opção permitida no enunciado. Assim, não inventamos os intervalos da fórmula perdida. Como os dois modelos são separados, a classe prevista e a gravidade estimada podem discordar. Não corrigimos essa diferença com uma regra inventada.

Conferimos o formato, os valores numéricos, os IDs e as classes de 1 a 4. Não retiramos valores extremos, preenchemos valores ausentes ou equilibramos a quantidade de casos por classe. O arquivo chamado "sem rótulo" repete os IDs e as entradas do outro arquivo; a diferença na gravidade é de arredondamento. Por isso, ele não é um teste novo. A Tabela 1 mostra que algumas classes têm muito mais exemplos que outras.

Tabela 1. Quantidade de registros por classe e divisão usada.

Classe	Total	Aprendizado	Observação
1. Crítico	234	141	93
2. Instável	832	499	333
3. Potencialmente estável	404	242	162
4. Estável	30	18	12
Total	1500	900	600

2.2. O que o programa deve fazer e como avaliamos

Os requisitos funcionais são receber os três sinais, estimar a gravidade, prever uma das quatro classes e comparar as previsões com as respostas conhecidas. Para o teste cego descrito na atividade, o programa recebe quatro colunas (ID e três sinais) e produz três (ID, gravidade e classe), sem cabeçalho, para cada técnica. Conferimos esse formato usando os registros reservados para avaliação. Não recebemos um arquivo cego novo.

A floresta precisa combinar as respostas das árvores. A MLP precisa colocar as entradas na escala usada no treino, calcular a saída e devolver a gravidade na escala original. Como requisitos não funcionais, buscamos conseguir repetir a execução no computador, registrar os parâmetros, acompanhar os cálculos e explicar os limites. As perguntas de uma árvore são mais fáceis de seguir, mas juntar várias árvores torna a explicação maior. Na rede, precisamos acompanhar pesos e operações numéricas. Para usar dados reais, também seria preciso proteger o acesso e permitir revisão da sugestão. Não implementamos esses controles nem garantimos um tempo de resposta para atendimento.

Para a gravidade, usamos RMSE: calculamos os erros ao quadrado, tiramos a média e depois a raiz. Quanto menor, menor o erro por esse critério:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i (\hat{g}_i - g_i)^2}.$$

Aqui, $\hat{g}_i$ é a gravidade prevista, g_i é a fornecida e n é a quantidade de registros. Para as classes, usamos acurácia (proporção de acertos), precisão, recall, F1 por classe e F1 macro. O recall dos críticos mostra quantos dos críticos foram reconhecidos. A precisão mostra quantos dos alertas críticos estavam certos. O F1 combina precisão e recall; o F1 macro é a média do F1 das quatro classes, com o mesmo peso para cada uma. Quando não há previsão de uma classe, registramos sua precisão e seu F1 como zero. A matriz de confusão mostra quais classes foram trocadas por quais.

Nenhuma medida resolve sozinha nossa preocupação. A acurácia pode parecer boa mesmo com muitos erros numa classe pequena. O recall pode aumentar junto com falsos alertas. Por isso, usamos as medidas em conjunto, embora a busca de configurações precise de um critério para escolher.

2.3. Separação dos dados e busca de ajustes

Mantivemos a divisão adotada na análise inicial: 900 linhas para aprender e 600 para avaliar depois. A divisão preserva aproximadamente a proporção das classes. Usamos semente 42 para repetir o sorteio. Isso não garante que o modelo funcione bem em dados novos. As 600 linhas não entram no treino nem na busca de configurações desta execução. Mas seus resultados já tinham sido vistos no estudo anterior. Por isso, não chamamos essa avaliação de teste cego ou de teste final nunca consultado.

Os slides de MLP apresentam hiperparâmetros, que são configurações escolhidas antes do treino, e recomendam validação cruzada [4]. Fizemos uma busca pequena com três partes dentro das 900 linhas, mantendo aproximadamente as proporções das classes. Em cada rodada, treinamos com duas partes e prevemos a terceira. Cada linha é avaliada uma vez por um modelo que não treinou com ela. Juntamos essas 900 previsões para comparar as configurações. Na MLP, calculamos a média e o desvio da padronização só com o treino de cada rodada.

Comparamos seis configurações por técnica e por tarefa. Foram 72 ajustes: duas técnicas, duas tarefas (gravidade e classe), seis configurações e três rodadas. Na floresta, variamos a profundidade máxima entre $\{3, 5, 7\}$ e a quantidade de árvores entre $\{9, 25\}$. Na MLP, variamos a quantidade de neurônios ocultos entre $\{4, 8, 12\}$ e a taxa de aprendizado entre $\{0{,}01; 0{,}05\}$. Escolhemos poucos valores para conseguir comparar e explicar o que foi feito. Isso não cobre todas as possibilidades, nem significa que o professor tenha exigido uma busca exaustiva.

Para a gravidade, escolhemos o menor RMSE. Para a classe, escolhemos o maior F1 macro, para dar o mesmo peso às quatro classes na comparação. Essa escolha não dá um peso extra aos críticos. Essa preocupação aparece depois, nos resultados por classe e nos cenários de custo. Em empate, fica a primeira configuração na ordem registrada, que começa pelos menores valores. Depois da escolha, treinamos novamente nas 900 linhas e avaliamos nas 600 reservadas. Não mudamos os valores da busca em função desses resultados.

2.4. Random Forest: as perguntas da árvore

Começamos pelas árvores porque é mais fácil entender o caminho até a resposta. Usamos uma versão simplificada de árvores CART. Cada nó faz uma pergunta: um atributo é menor ou igual a determinado valor? Para classificar, usamos Gini, $G = 1 - \sum_c p_c^2$, em que p_c é a proporção da

classe no nó. Para estimar gravidade, usamos a variação dos valores em torno da média. Esses critérios aparecem nos slides, que também distinguem CART, ID3 e C4.5 [2].

O ganho de um corte é a impureza antes da divisão menos a média das impurezas depois, ponderada pelo tamanho dos grupos. Se os dois lados continuam com as mesmas proporções de classes, a pergunta não reduziu a mistura. Isso não prova que todas as outras perguntas sejam ruins. Entropia e ganho de informação ajudaram a entender essa ideia no estudo, mas o código deste trabalho usa Gini [3].

Em cada nó, sorteamos dois dos três atributos. Para cada atributo, examinamos até 12 pontos médios entre valores diferentes. Se houver mais candidatos, escolhemos posições igualmente espaçadas na lista ordenada. Ficamos com o maior ganho positivo entre os cortes examinados. Esse é o corte "melhor" segundo o critério naquele momento, sem garantia de formar a melhor árvore inteira.

Ao treinar com as 900 linhas, cada árvore recebe 900 sorteios com reposição: uma linha pode aparecer mais de uma vez e outra pode ficar de fora. Isso e o sorteio de atributos ajudam a formar árvores diferentes, mas não as tornam independentes. Na gravidade, usamos a média das previsões. Na classe, calculamos a média das proporções de classe das folhas alcançadas e escolhemos a maior. Portanto, não contamos apenas o voto da classe vencedora de cada árvore. A regra de custo, quando usada, vem depois dessas probabilidades.

Mantemos pelo menos oito entradas por folha. A árvore também para quando o nó fica puro, chega à profundidade máxima ou não encontra ganho positivo nos cortes permitidos. Não fazemos poda depois do treino. Esses limites procuram evitar que a árvore se prenda demais aos exemplos. Esse problema é o sobreajuste: aprender detalhes do treino que não ajudam em outros dados. Não existe uma profundidade que seja sempre boa; por isso, comparamos as configurações fora do treino de cada rodada.

2.5. MLP: como a rede aprende

A MLP recebe os três sinais padronizados: de cada valor, subtraímos a média e dividimos pelo desvio do treino, $z = (x - \mu)/\sigma$. Usamos uma camada oculta com função tanh. Para a gravidade, a saída é linear e o alvo também é padronizado; no fim, voltamos à escala original. Para a classe, temos quatro saídas que a função softmax transforma em probabilidades estimadas. As duas redes têm seus próprios pesos. Usamos os mecanismos apresentados nos materiais de redes [4].

Os pesos começam com valores aleatórios em intervalos simétricos definidos pelo tamanho das camadas, com semente 42. Os vieses começam em zero. Fazemos 800 atualizações, usando todos os exemplos de treino em cada uma. A retropropagação calcula como cada peso afeta a perda. A descida do gradiente usa esse cálculo para ajustar os pesos: $w \leftarrow w - \eta\, \partial L/\partial w$, em que η é a taxa de aprendizado. Na regressão, a perda é metade do erro quadrático médio do alvo padronizado; na classificação, é entropia cruzada, que penaliza probabilidades baixas para a classe correta.

Nas duas redes, acrescentamos a penalização L2, $\lambda(\|W_1\|^2 + \|W_2\|^2)/2$, com $\lambda = 0{,}001$. Ela penaliza pesos grandes, mas não os vieses, e não garante bom resultado em dados novos. Mantivemos uma camada pequena e 800 passos para conseguir acompanhar a implementação. A busca muda só neurônios e taxa. Parar em 800 passos não prova que chegamos à melhor solução ou que a rede parou de melhorar. Se o resultado for ruim, pode haver problema na representação, na configuração ou no treino. Isso não permite dizer que toda MLP é ruim.

2.6. Custo dos erros: uma escolha nossa

Consideramos mais grave deixar uma vítima crítica sem reconhecimento. Mas não sabemos o custo real de cada erro, nem o tempo e as equipes necessários para cada atendimento. Por isso, usamos fatores $k \in \{1, 2, 5, 10\}$ para ver o que muda quando damos mais peso a esse erro. São hipóteses de comparação, não valores médicos ou estimativas de vidas perdidas.

Na matriz C, acertar custa zero. Prever outra classe para uma vítima crítica custa k. Qualquer outro erro custa um. Para cada classe possível, multiplicamos a probabilidade de cada classe real pelo custo de escolher aquela resposta e somamos. Ficamos com a resposta de menor custo esperado:

$$\hat{a}(x) = \arg \min_{j \in \{1,2,3,4\}} \sum_{i=1}^{4} \hat{p}(i \mid x) C_{ij}. \tag{1}$$

Aqui, i é a classe real possível e j é a resposta que estamos considerando. Com $k = 1$, isso equivale a escolher a maior probabilidade. Em empate, fica a primeira classe. Mudar k muda só a decisão feita com as probabilidades: não treinamos novamente os modelos. Depois, calculamos o custo observado usando as classes fornecidas nos dados. Esse custo pode ser diferente do esperado pelas probabilidades.

Essa escolha simplifica bastante o problema. Todos os erros com vítimas não críticas recebem o mesmo custo. Além disso, decidimos uma linha de cada vez, sem calcular o efeito de ocupar uma equipe que poderia atender outra pessoa. A matriz ajuda a comparar nossa prioridade, mas não resolve a organização do resgate.

3. Resultados: o que encontramos

3.1. Antes e depois dos ajustes

A Tabela 2 mostra as configurações escolhidas usando só as 900 linhas. A vencedora de cada tarefa é a melhor entre as seis testadas, segundo o critério definido. Os outros parâmetros continuam iguais. A configuração para gravidade pode ser diferente daquela para classe porque as tarefas e os critérios são diferentes.

Tabela 2. Configurações escolhidas pela validação cruzada nas 900 linhas.

Técnica	Tarefa	Configuração selecionada	Validação
Floresta	Regressão	25 árvores; profundidade 7	RMSE 5,345
Floresta	Classificação	9 árvores; profundidade 7	F1 macro 0,774
MLP	Regressão	8 neurônios; taxa 0,05	RMSE 8,399
MLP	Classificação	8 neurônios; taxa 0,05	F1 macro 0,394

Começamos com nove árvores de profundidade máxima cinco e uma MLP com oito neurônios, taxa 0,05 e 800 passos. A Tabela 3 compara essa versão com as escolhidas. Também colocamos uma referência simples: prever sempre a média da gravidade do treino e a classe mais frequente. Isso ajuda a ver se os modelos aprenderam algo além dessa resposta repetida.

Tabela 3. Antes e depois dos ajustes, com custos iguais. Menor RMSE significa menos erro; acerto e F1 macro maiores são melhores nesses critérios.

Técnica	Versão	Conjunto	RMSE	Acerto (%)	F1 macro
Floresta	Inicial	Treino	5,452	89,33	0,671
MLP	Inicial	Treino	8,201	63,11	0,389
Floresta	Inicial	Observação	6,461	85,83	0,638
MLP	Inicial	Observação	8,521	63,67	0,395
Floresta	Selecionada	Treino	3,638	93,00	0,904
MLP	Selecionada	Treino	8,201	63,11	0,389
Floresta	Selecionada	Observação	4,678	88,50	0,754
MLP	Selecionada	Observação	8,521	63,67	0,395
Referência	Constante	Observação	16,490	55,50	0,178

Nos 600 registros, a floresta escolhida teve RMSE 4,678, acurácia 88,50% e F1 macro 0,754; a MLP obteve, respectivamente, 8,521, 63,67% e 0,395. A busca manteve a configuração inicial das duas MLPs. Na floresta, escolheu profundidade sete, com 25 árvores para gravidade e nove para classe. A escolha veio da comparação na validação cruzada; aumentar a profundidade não garante melhorar. A diferença entre treino e avaliação merece atenção, mas sozinha não prova sobreajuste.

A Tabela 4 separa os resultados por classe, com custos iguais. Nos 600 registros, há 93 críticos e só 12 estáveis. Um acerto a mais entre os estáveis já muda seu recall em cerca de 8,33 pontos percentuais. Precisamos lembrar desse número pequeno antes de tirar uma conclusão pela porcentagem.

Tabela 4. Resultados por classe nos 600 registros, com custos iguais.

Técnica	Classe	Casos	Precisão (%)	Recall (%)	F1
Floresta	1	93	97,10	72,04	0,827
Floresta	2	333	87,81	95,20	0,914
Floresta	3	162	86,23	88,89	0,875
Floresta	4	12	100,00	25,00	0,400
MLP	1	93	72,00	19,35	0,305
MLP	2	333	64,40	85,29	0,734
MLP	3	162	59,70	49,38	0,541
MLP	4	12	0,00	0,00	0,000

A floresta reconheceu 67 dos 93 críticos e 3 dos 12 estáveis. A MLP reconheceu 18 dos críticos e 0 dos estáveis. Na versão inicial, nenhuma das duas reconhecia os estáveis. A floresta passou a reconhecer alguns; a MLP continuou sem reconhecer essa classe. Na Figura 1, os números fora da diagonal mostram quais classes foram confundidas. Olhar só a porcentagem geral de acertos esconderia essas diferenças. A busca também trouxe um resultado que exige cuidado: a floresta escolhida deixou de reconhecer 26 críticos, contra 24 na versão inicial. Melhorar o F1 macro não melhorou o reconhecimento dos críticos nessa avaliação. Isso mostra uma limitação do critério que usamos para escolher.

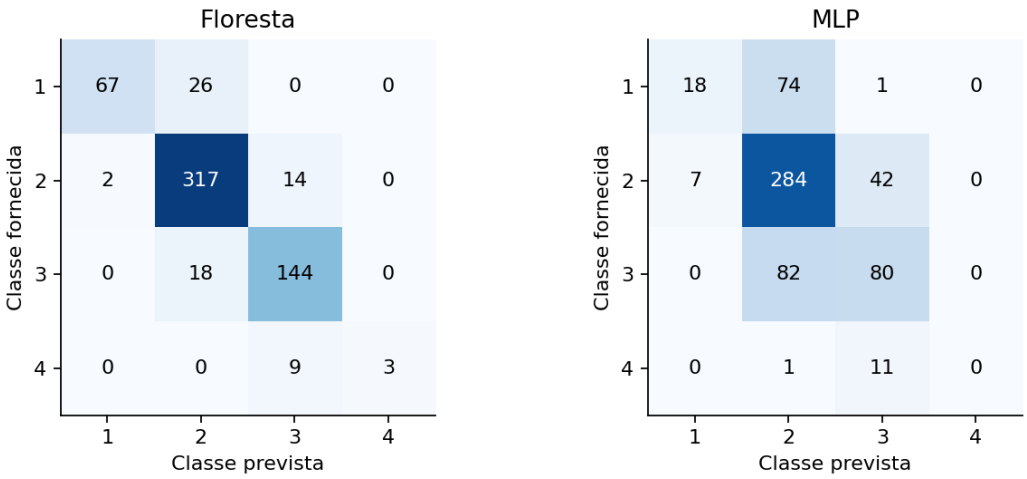


Figura 1. Acertos e erros dos modelos escolhidos nos 600 registros, com custos iguais. Linhas: classe fornecida. Colunas: classe prevista.

3.2. O que muda quando damos mais peso aos críticos

Na Tabela 5, FN é a quantidade de críticos não reconhecidos. FP é a quantidade de vítimas de outras classes que receberam um alerta crítico. Comparamos o custo da decisão por custo esperado com o da decisão pela maior probabilidade, usando o mesmo k em cada linha. Não comparamos diretamente o custo total de linhas com valores diferentes de k, porque mudamos o peso dos erros.

Tabela 5. Cenários nos 600 registros. Última coluna: custo pela decisão de menor custo esperado / pela maior probabilidade, usando o mesmo fator.

Técnica	k	FN	FP	Recall (%)	Precisão (%)	Custo total
Floresta	1	26	2	72,04	97,10	69 / 69
Floresta	2	17	9	81,72	89,41	84 / 95
Floresta	5	4	34	95,70	72,36	94 / 173
Floresta	10	1	76	98,92	54,76	125 / 303
MLP	1	75	7	19,35	72,00	218 / 218
MLP	2	48	32	48,39	58,44	264 / 293
MLP	5	25	138	73,12	33,01	381 / 518
MLP	10	18	267	80,65	21,93	523 / 893

Na floresta, passar de $k = 1$ para $k = 5$ reduziu os críticos não reconhecidos de 26 para 4, enquanto os falsos alertas aumentaram de 2 para 34. Para $k = 10$, a quantidade de críticos não reconhecidos foi 1, com 76 falsos alertas. Por isso, não escolhemos olhando só o recall. Na MLP, $k = 5$ deixou 25 críticos sem reconhecimento e produziu 138 falsos alertas. A floresta teve menor custo observado que a MLP nos quatro fatores considerados. A floresta foi melhor pelo custo nesses cenários. Isso não permite afirmar que ela seja sempre melhor.

Reduzir esse custo significa melhorar segundo os valores que escolhemos. Não prova que o atendimento real seria melhor. Também não conferimos se as probabilidades correspondem às frequências de acerto em outra população, o que seria uma avaliação de calibração. A decisão depende dessas probabilidades e dos custos. Não escolhemos um valor de k definitivo para um resgate real, pois faltam informações para justificar isso.

3.3. Semelhanças, diferenças e limites

As duas técnicas usam as mesmas entradas e divisões dos dados. Aprendem com exemplos que já têm resposta e dependem dos parâmetros. A floresta faz divisões e combina árvores. A MLP ajusta pesos em conjunto para transformar as entradas em saídas. É mais fácil seguir as perguntas de uma árvore do que acompanhar todos os pesos de uma rede, mas entender o caminho não garante que a previsão esteja certa.

Usamos seis configurações e as mesmas três partes de validação para cada técnica. Isso não significa o mesmo custo de execução: treinar árvores e redes exige operações diferentes. Não medimos tempo ou memória para dizer qual foi mais eficiente. Trabalhamos com uma semente, poucos cortes nas árvores e 800 passos nas redes. A comparação vale para essas condições, não para todas as versões possíveis de Random Forest e MLP.

4. O que fica de fora e quem decide

Nossa escolha é usar o modelo para sugerir, com a decisão final humana. Isso também tem limite: uma pessoa pode confiar demais na resposta e repetir o erro do sistema. Seria preciso permitir revisão, mostrar as informações usadas e registrar correções. Só colocar alguém no final do processo não garante uma boa decisão.

Na floresta, seguir as perguntas de uma árvore pode ajudar o profissional a explicar e questionar uma sugestão, embora juntar várias árvores dificulte essa revisão. Na MLP, não temos esse caminho de perguntas; mostrar uma probabilidade pode dar uma impressão de certeza maior do que os resultados permitem. Nas duas técnicas, falsos alertas podem aumentar o trabalho da equipe. O sistema deve ajudar a discutir a prioridade, sem transformar sua resposta numa ordem que o profissional não possa contestar.

Um crítico não reconhecido pode ter o socorro atrasado. Um falso alerta pode ocupar uma equipe necessária a outra vítima. Foi por isso que comparamos os custos, mas nossa matriz não calcula duração dos atendimentos, deslocamento ou disponibilidade de equipes. Prever uma classe e organizar uma sequência de resgates exigem informações diferentes.

Também não podemos afirmar que o modelo seja justo só porque não usamos atributos sociais. Não temos informações para comparar grupos de pessoas ou saber quem ficou de fora da coleta. Uma classe com resultado pior pode ter poucos exemplos ou ser difícil de separar com esses sinais; isso não é uma característica das pessoas daquela classe. Se usássemos dados reais, precisaríamos limitar o acesso e explicar para que seriam usados. O programa deste exercício roda localmente, sem enviar sinais a serviços externos.

A falta de contexto se relaciona à discussão mais ampla do *frame problem*: o que considerar e o que ficou de fora da representação. Mas a formulação clássica trata de representar e inferir o que muda e o que permanece depois de uma ação [5]. Nosso classificador não representa ações nem mudanças no tempo. Ele não resolve esse problema. A pessoa que decidir também terá que lidar com a falta de informações, sem garantia de resolver tudo.

5. Conclusão

Usamos as ferramentas da disciplina para comparar duas técnicas e explicar nossas escolhas. Separamos treino e avaliação, limitamos os ajustes testados e olhamos os erros por classe. A busca automatizada ajudou a comparar configurações, mas o que comparar e como escolher continuaram sendo decisões de modelagem.

Para nós, dizer que um modelo “acerta mais” não basta. Precisamos olhar quais erros ele comete e o peso que estamos dando a eles. Nos cenários analisados, reconhecer mais críticos veio

junto com mais falsos alertas. Mantivemos esses limites na análise, inclusive quando o resultado foi fraco. Os três sinais ajudam a produzir uma sugestão, mas continuam insuficientes para decidir sozinhos um resgate.

6. Participação e ferramentas de apoio

Repositório do projeto: https://github.com/mariocjun/sistemas-inteligentes.

Pasta desta entrega: `tarefas/tarefa-02-entrega`. Ela reúne o notebook executado, os dados e os resultados. O artigo em PDF é enviado diretamente no Moodle.

Mário Cordeiro Júnior e André F. Maccarini pensaram juntos na modelagem, na implementação e na análise, sem separar uma etapa exclusiva para cada um. Cada participante dedicou aproximadamente 10 horas, totalizando 20 horas da equipe.

Utilizamos o Codex, assistente de inteligência artificial da OpenAI. A ferramenta ajudou a elaborar e revisar códigos, explicar cálculos, conferir resultados e reescrever o texto. Também deu apoio às configurações iniciais de exemplo e à execução da busca de ajustes descrita no trabalho. As escolhas sobre o objetivo, os riscos, a decisão humana e os limites foram discutidas pela equipe. A revisão e a responsabilidade pelo conteúdo são nossas.

Esse apoio ocorreu no estudo inicial e na preparação da entrega. Incorporamos código gerado com esse apoio nas funções dos modelos e nas etapas de comparação e exportação reunidas no notebook, além das tabelas e dos trechos reescritos. Conferimos os números pela execução e pelas verificações descritas abaixo. A ajuda com código e escrita não comprova que as escolhas sejam adequadas a um resgate real, nem substitui a compreensão e a revisão dos participantes.

Conferimos as medidas, a separação dos dados e o formato dos arquivos de previsão. Também conferimos se as probabilidades somam um e se os gradientes calculados pela rede concordam com uma aproximação numérica em exemplos pequenos. Isso ajuda a verificar o código, mas não prova que o modelo seja suficiente para um atendimento real.

Referências

[1] UTFPR. *Tarefa 2: Classificação – Resgate de Vítimas de Catástrofes Naturais, Desastres ou Grandes Acidentes*. Enunciado e instruções da atividade ICSI30 no Moodle, materiais fornecidos na disciplina. Cópias locais consultadas em novembro de 2026.

[2] Giménez-Lugo, G. A. *Trees and Random Forests*. Slides disponibilizados em Sistemas Inteligentes, UTFPR, arquivo 04_B_TREES_RandomForests.pdf, especialmente slides 3–5 e 8–10.

[3] Tacla, C. A. *Aprendizado Simbólico*. Slides disponibilizados em Sistemas Inteligentes, UTFPR, arquivo 03_AprendizadoSimbolico_completo.pdf, especialmente slides 40–47.

[4] Tacla, C. A. *Redes Neurais: Multilayer Perceptron*. Slides disponibilizados em Sistemas Inteligentes, UTFPR, arquivo 02_C_TACLA_MLP-v4.pdf, especialmente slides 3, 52 e 58–60.

[5] UTFPR. *Frame Problem*. Material disponibilizado em Sistemas Inteligentes, arquivo 03_FRAME_PROBLEM.pdf, páginas 2–6.